Universidad de Santiago de Chile
Departamento de Ingeniería Informática
Fundamentos de Ingeniería de Software

Laboratorio 1

Definición del proyecto y planificación inicial

SentinelaTI

Plataforma de monitoreo y mantención remota de computadores para equipos de TI

Integrantes: Felipe Cifuentes, Paula Rivas

Profesor: Joaquín Macías Cáceres

Repositorio: https://github.com/gazpel/SentinelaTI

Fecha: 24-09-2026

Índice

1. Definición del problema y alcance 3
1.1 Contexto y problema 3
1.2 Objetivos 3
1.3 Solución propuesta 3
1.4 Parámetros monitoreados 3
1.5 Alcance 4
1.6 Verificación del checklist de viabilidad 4
2. Análisis del contexto socio-técnico 5
2.1 Actores y stakeholders 5
2.2 Propiedades emergentes esperadas 6
2.3 Factores humanos y organizacionales 6
3. Modelo de proceso de desarrollo 7
4. Propuesta preliminar de stack tecnológico 7
5. Planificación inicial 8
5.1 Estructura de división del trabajo (WBS) 8
5.2 Carta Gantt 9
5.3 Hitos verificables 9
5.4 Roles del equipo 10
6. Identificación preliminar de riesgos 10

1. Definición del problema y alcance

1.1 Contexto y problema

En empresas medianas, el área de TI administra decenas o cientos de computadores repartidos entre distintas áreas y, a veces, en teletrabajo. Hoy la mantención suele ser **reactiva**: TI se entera de un problema cuando el usuario levanta un ticket o llama por teléfono (disco lleno, equipo sobrecalentado, sistema sin parches de seguridad, software no autorizado). Revisar cada equipo exige ir físicamente al puesto, lo que consume horas del técnico e interrumpe al trabajador.

No existe una vista centralizada del estado del parque computacional ni un registro histórico que permita anticiparse a fallas. Tampoco hay forma de ejecutar acciones de mantención (reiniciar, actualizar, instalar software) sin intervenir presencialmente.

1.2 Objetivos

Objetivo general: desarrollar un sistema que permita al equipo de TI de una organización monitorear de forma centralizada el estado de sus computadores y ejecutar acciones de mantención de forma remota.

Objetivos específicos:

- Recolectar automáticamente, mediante un agente instalado en cada equipo, información de estado, hardware y software.
- Presentar el estado actual e histórico de los equipos en un panel web (dashboard) con alertas ante valores críticos.
- Permitir a TI enviar comandos remotos a los equipos, con control de permisos por rol y registro de auditoría.
- Garantizar que el agente no pueda ser desinstalado ni detenido por el usuario final.

1.3 Solución propuesta

SentinelaTI se compone de tres partes:

- **Agente:** programa liviano que TI instala en cada computador. Corre como servicio del sistema operativo, recolecta métricas cada cierto intervalo, las envía al servidor y consulta si hay comandos pendientes para ejecutar.
- **Servidor (API):** recibe y almacena las métricas, evalúa umbrales para generar alertas, gestiona la cola de comandos remotos, los usuarios y la auditoría.
- **Panel web:** interfaz para administradores y técnicos de TI donde se ve el listado de equipos, su detalle e historial, las alertas, y desde donde se ordenan las acciones remotas.

1.4 Parámetros monitoreados

Se definieron 9 indicadores para la mantención preventiva que se obtienen con el agente.

Parámetro	Uso para TI
Estado de actualizaciones del SO	Detectar equipos sin parches de seguridad.
Aplicaciones instaladas	Inventario de software, licencias y detección de software no autorizado.
Horas encendido (uptime)	Detectar equipos que no se reinician hace semanas o que no se apagan.
Historial de temperaturas	Anticipar fallas por sobrecalentamiento (ventiladores sucios, pasta térmica).

Parámetro	Uso para TI
Uso de almacenamiento	Alertar antes de que el disco se llene.
Uso de CPU y RAM	Identificar equipos subdimensionados que requieren recambio.
Salud del disco SMART y batería	Predecir fallas de disco y degradación de baterías en notebooks.
Estado de antivirus y firewall	Verificar la postura de seguridad básica del equipo.
Inventario de hardware: modelo, n° de serie, IP, último usuario	Identificar y ubicar cada equipo; apoyo a la gestión de activos.

Tabla 1. Parámetros que recolecta el agente.

1.5 Alcance

El proyecto se desarrolla durante un semestre, por lo que se distingue entre lo que se especifica y modela (proyecto completo) y lo que se construye en el prototipo funcional (L5).

Categoría	Detalle
Incluido en el prototipo (implementación real)	Instalación y registro del agente en Windows 10/11; recolección y envío de métricas; dashboard con listado, detalle e historial; alertas por umbral; usuarios con roles; acciones remotas apagar/reiniciar e instalar aplicación desde un catálogo aprobado; registro de auditoría.
Especificado y simulado en el prototipo	Actualización remota del SO, formateo remoto y navegación/escritorio remoto. Aparecen en el panel y siguen el mismo flujo de comando y auditoría, pero el agente solo registra la orden y simula su ejecución, sin efecto real sobre el equipo.
Fuera del alcance	Soporte para macOS/Linux, aplicación móvil, integración con directorios corporativos (Active Directory), multiempresa/facturación y despliegue masivo automatizado del agente.

Tabla 2. Alcance del proyecto.

Todas las pruebas de acciones remotas se realizarán sobre **máquinas virtuales**, nunca sobre los equipos personales de los integrantes.

1.6 Verificación del checklist de viabilidad

Criterio	¿Cumple?	Evidencia
Dominio comprensible sin conocimiento experto	Sí	Los integrantes conocen el funcionamiento de un computador y el trabajo de soporte de TI; las reglas del negocio (umbrales, permisos, comandos) son definibles por el propio equipo.
2 a 4 tipos de actores distintos	Sí	Administrador de TI, Técnico de TI, Usuario final y el Agente como actor de sistema (sección 2.1).
8 a 15 requerimientos funcionales	Sí	Se identifican 15 RF preliminares (Tabla 4).
Al menos 3 requerimientos no funcionales relevantes	Sí	Seguridad, protección contra desinstalación, bajo impacto en rendimiento, tolerancia a desconexión, privacidad y usabilidad (Tabla 5).
Modelo de clases significativo (5 a 8 clases)	Sí	Clases candidatas: Usuario, Equipo, Agente, LecturaMétrica, AplicaciónInstalada, Actualización, ComandoRemoto, UmbralAlerta, Alerta, RegistroAuditoría.
Alcance implementable parcialmente	Sí	El prototipo abarca monitoreo y dos acciones remotas reales; las acciones de mayor riesgo se simulan (Tabla 2).

Tabla 3. Checklist de viabilidad del enunciado del proyecto.

Requerimientos funcionales preliminares (se refinarán en L2):

ID	Requerimiento	Actor
RF01	Registrar un equipo en la plataforma al instalar el agente con un código de la organización.	Admin TI / Agente
RF02	Recolectar y enviar periódicamente métricas: CPU, RAM, almacenamiento, temperatura y horas encendido.	Agente
RF03	Reportar el inventario de hardware y de aplicaciones instaladas.	Agente
RF04	Reportar el estado de actualizaciones del sistema operativo.	Agente
RF05	Listar todos los equipos con su estado actual, con búsqueda y filtros.	Admin / Técnico
RF06	Ver el detalle e historial de métricas de un equipo en gráficos.	Admin / Técnico
RF07	Configurar umbrales y generar alertas cuando se superan.	Admin TI
RF08	Enviar orden remota de apagar o reiniciar un equipo.	Admin / Técnico
RF09	Instalar remotamente una aplicación desde un catálogo aprobado.	Admin / Técnico
RF10	Ordenar la actualización remota del sistema operativo (simulada).	Admin / Técnico
RF11	Ordenar el formateo remoto de un equipo con doble confirmación (simulado).	Admin TI
RF12	Iniciar una sesión de navegación/escritorio remoto (simulada).	Admin / Técnico
RF13	Gestionar usuarios del panel y sus roles.	Admin TI
RF14	Registrar en una bitácora de auditoría toda acción remota (quién, qué, cuándo, resultado).	Sistema
RF15	Notificar al usuario final en su equipo antes de ejecutar una acción remota que lo afecte.	Agente / Usuario final

Tabla 4. Requerimientos funcionales preliminares.

ID	Tipo	Requerimiento no funcional
RNF01	Seguridad	La comunicación agente–servidor usa HTTPS y cada agente se autentica con un token propio. Solo el rol Administrador puede ordenar un formateo.
RNF02	Protección	El agente corre como servicio del sistema y no puede ser detenido ni desinstalado por una cuenta sin privilegios de administrador.
RNF03	Rendimiento	El agente no supera en promedio el 2% de CPU ni 100 MB de RAM en el equipo monitoreado.
RNF04	Disponibilidad	Si el equipo pierde conexión, el agente guarda las métricas localmente y las envía al reconectarse.
RNF05	Privacidad	El agente no recolecta contenido personal (archivos, correos ni historial de navegación).
RNF06	Usabilidad	Un técnico identifica los equipos en estado crítico desde la pantalla principal, sin navegar a otra vista.

Tabla 5. Requerimientos no funcionales preliminares.

2. Análisis del contexto socio-técnico

SentinelaTI no es solo software: se inserta en una organización con personas, procesos y políticas. Su éxito depende tanto de la tecnología como de cómo lo acepten TI y los trabajadores.

2.1 Actores y stakeholders

Actor / stakeholder	Tipo	Necesidades e intereses
Administrador de TI	Usuario directo	Visión global del parque, configurar umbrales y catálogo de aplicaciones, gestionar usuarios, autorizar acciones críticas (formateo).
Técnico de TI	Usuario directo	Detectar rápido equipos con problemas, ver su historial y resolver a distancia (reiniciar, instalar, actualizar) sin ir al puesto.
Usuario final (trabajador)	Usuario indirecto	Que su equipo funcione bien, ser avisado antes de acciones que interrumpan su trabajo y que no se invada su privacidad.
Agente	Actor de sistema	Reporta métricas y ejecuta los comandos recibidos.
Jefatura de TI / Gerencia	Stakeholder (no usuario)	Reportes de estado del parque, reducción de costos de soporte, cumplimiento de políticas de seguridad.
Área legal / RR.HH.	Stakeholder (no usuario)	Que el monitoreo respete la normativa de protección de datos personales y esté informado en las políticas internas.

Tabla 6. Actores y stakeholders.

2.2 Propiedades emergentes esperadas

Son propiedades del sistema completo (agente + servidor + panel + personas), que no pertenecen a ninguna parte por separado.

Funcionales:

- **Visibilidad centralizada del parque:** solo emerge al combinar agentes en muchos equipos con el servidor y el panel.
- **Mantención preventiva:** la combinación de historial y alertas permite actuar antes de la falla, cosa que ningún componente logra por sí solo.

No funcionales:

- **Seguridad:** depende del agente, la red, la API, los roles y las contraseñas de las personas. Es crítica porque el sistema puede apagar o formatear equipos: una cuenta comprometida afecta a toda la organización.
- **Fiabilidad:** un comando debe ejecutarse una sola vez y en el equipo correcto, aunque haya cortes de red.
- **Rendimiento:** el agente no debe entorpecer el trabajo del usuario.
- **Aceptabilidad y privacidad:** si los trabajadores lo perciben como vigilancia, pueden resistirse o intentar evadirlo.

2.3 Factores humanos y organizacionales

Tipo de cambio	Impacto esperado
Cambios en el proceso	La mantención pasa de reactiva (por ticket o llamada) a preventiva (por alertas). TI debe definir umbrales, un catálogo de aplicaciones aprobadas y un procedimiento de aprobación para acciones críticas.
Cambios en el trabajo	Los técnicos hacen menos visitas presenciales y más trabajo remoto y de análisis, lo que requiere capacitación. El usuario final ya no instala software por su cuenta y puede recibir reinicios programados.
Cambios en la estructura de poder	TI gana control sobre equipos que antes manejaba cada usuario, lo que puede generar resistencia o sensación de vigilancia. Se mitiga con una política de uso informada (qué se recolecta y qué no), notificaciones antes de cada acción y auditoría que también controla a TI.

Tabla 7. Factores humanos y organizacionales.

3. Modelo de proceso de desarrollo

Se elige un modelo **ágil basado en Scrum**, con entregas incrementales alineadas a los laboratorios del curso. La justificación responde a las preguntas del enunciado:

Pregunta	Respuesta para SentinelaTI
¿Qué tan bien se comprenden los requerimientos?	Parcialmente. El objetivo general es claro, pero detalles como qué umbrales usar, qué métricas son realmente útiles o cómo notificar al usuario se descubrirán al construir y mostrar avances.
¿Es probable que cambien?	Sí. El propio enunciado indica que el proyecto puede ir modificándose durante el semestre, y la factibilidad técnica de algunas funciones (temperatura, protección contra desinstalación) puede obligar a ajustar el alcance.
¿Se necesitan avances tempranos?	Sí. El curso exige entregas cada 2 a 3 semanas (L1 a L7) y un prototipo funcional en L5; conviene validar temprano que el agente puede leer las métricas.
¿Qué tamaño y complejidad tiene?	Proyecto pequeño: equipo de 2 personas y 17 semanas, con complejidad moderada (tres componentes distribuidos). Un proceso liviano es más adecuado que uno fuertemente dirigido por planes como RUP o cascada; el equipo reducido exige además priorizar bien el alcance en cada sprint.

Tabla 8. Justificación del modelo de proceso.

Adaptación de Scrum al curso:

- **Sprints** de 2 a 3 semanas, cada uno terminando en la entrega de un laboratorio (ver Gantt).
- **Roles:** Product Owner (prioriza el backlog), Scrum Master (facilita y controla plazos) y equipo de desarrollo. Los roles pueden rotar por entrega.
- **Artefactos:** Product Backlog y Sprint Backlog en GitHub Projects; incremento = documento técnico actualizado y, desde L4, código funcional.
- **Ceremonias livianas:** planificación al inicio de cada sprint, reunión breve de seguimiento dos veces por semana y revisión/retrospectiva al entregar.

Como el curso exige un documento técnico incremental con trazabilidad, el proceso combina Scrum con documentación suficiente en cada sprint. No es un enfoque ágil sin documentación.

4. Propuesta preliminar de stack tecnológico

Componente	Tecnología	Justificación
Agente	Python 3 + psutil, WMI; instalado como servicio de Windows	psutil entrega CPU, RAM, disco y uptime con pocas líneas; WMI da acceso a actualizaciones, aplicaciones y temperatura. Correr como servicio permite cumplir RNF02.
Servidor / API	Python + FastAPI + SQLAlchemy	Mismo lenguaje que el agente, lo que reduce la curva de aprendizaje; FastAPI genera documentación de la API automáticamente.
Base de datos	PostgreSQL (SQLite en desarrollo)	Base relacional conocida por el equipo, adecuada para historial de métricas y auditoría.
Panel web	React + Vite + librería de gráficos (Recharts)	Permite construir el dashboard y los gráficos de historial de forma modular.
Comunicación	API REST sobre HTTPS; el agente consulta periódicamente sus comandos pendientes	No requiere abrir puertos en los equipos cliente: simple y más seguro.

Componente	Tecnología	Justificación
Gestión y apoyo	GitHub + GitHub Projects, draw.io/PlantUML, pytest, máquinas virtuales Windows	Control de versiones obligatorio, tablero Scrum, diagramas UML, pruebas y entorno seguro para probar acciones remotas.

Tabla 9. Stack tecnológico preliminar.

Manejabilidad por el equipo: Python es el lenguaje que mejor maneja el equipo, y se usa tanto en el agente como en el servidor, de modo que la mayor parte del sistema se escribe en un solo lenguaje. React es la tecnología con menos experiencia previa; se abordará con trabajo en parejas y limitando la interfaz a pocas vistas. La plataforma objetivo se limita a Windows 10/11 para reducir la complejidad.

5. Planificación inicial

5.1 Estructura de división del trabajo (WBS)

La WBS se organiza por fases del ciclo de vida, que coinciden con los sprints y laboratorios. Cada paquete de trabajo termina en un entregable verificable.

Código	Paquete de trabajo	Entregable
1	**Gestión del proyecto**	
1.1	Configurar repositorio y tablero Scrum	Repositorio GitHub y tablero GitHub Projects
1.2	Reuniones de planificación, seguimiento y retrospectiva	Actas breves en el repositorio
1.3	Seguimiento de riesgos	Tabla de riesgos actualizada en cada entrega
2	**Definición y planificación (L1)**	
2.1	Definir problema, objetivos, alcance y checklist	Sección 1 del documento
2.2	Analizar contexto socio-técnico	Sección 2 del documento
2.3	Elegir modelo de proceso y stack	Secciones 3 y 4 del documento
2.4	Elaborar WBS, Gantt y riesgos	Secciones 5 y 6 del documento
3	**Requerimientos (L2)**	
3.1	Especificar RF y RNF	Documento de requerimientos
3.2	Casos de uso y diagrama de casos de uso	Diagrama y especificaciones
3.3	Product Backlog con historias de usuario priorizadas	Backlog en GitHub Projects
4	**Modelado (L3)**	
4.1	Diagrama de clases del dominio	Diagrama UML de clases
4.2	Diagramas de secuencia de los casos de uso principales	Diagramas UML de secuencia
5	**Diseño (L4)**	
5.1	Arquitectura agente–servidor–panel	Diagrama de componentes y de despliegue
5.2	Diseño OO detallado y modelo de datos	Diagrama de clases de diseño y modelo de BD
5.3	Maquetas de interfaz (wireframes)	Wireframes del panel
5.4	Prueba técnica del agente (spike)	Script que lee métricas en una VM Windows

Código	Paquete de trabajo	Entregable
6	**Implementación y UI (L5)**	
6.1	Agente: recolección, envío y ejecución de comandos	Agente instalable como servicio
6.2	API: métricas, alertas, comandos, usuarios y auditoría	API funcionando
6.3	Panel web: listado, detalle, alertas y acciones	Dashboard funcionando
7	**Pruebas y documentación (L6)**	
7.1	Plan y casos de prueba trazados a requerimientos	Plan de pruebas
7.2	Ejecución de pruebas y registro de resultados	Informe de pruebas
7.3	Integración del documento técnico final	Documento integrado
8	**Presentación final (L7)**	
8.1	Preparar presentación y demostración	Presentación y demo

Tabla 10. WBS del proyecto.

5.2 Carta Gantt

Escala en semanas del semestre (semana 3 a 17). La semana 16 está reservada para la PEP2. La fase 1 (gestión) es transversal.

Actividad (WBS)	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1 Gestión del proyecto (Scrum)															
2 Sprint 0 – Definición y planificación (L1)															
3 Sprint 1 – Requerimientos (L2)															
4 Sprint 2 – Modelado UML (L3)															
5 Sprint 3 – Arquitectura y diseño (L4)															
5.4 Prueba técnica del agente (spike)															
6 Sprint 4 – Implementación y UI (L5)															
7 Sprint 5 – Pruebas y documentación (L6)															
PEP2 (sin laboratorio)															
8 Sprint 6 – Presentación final (L7)															
Hitos		◆1	◆2		◆3		◆4			◆5	◆6	◆7	◆8		◆9

Figura 1. Carta Gantt del proyecto. ◆ indica un hito (ver Tabla 11).

5.3 Hitos verificables

Hito	Semana	Criterio de verificación
H1	4	Repositorio creado en GitHub con README, estructura de carpetas y acceso de todos los integrantes; tablero Scrum creado.
H2	5	Documento L1 subido a UVirtual el 24-09-2026 antes de las 19:00.
H3	7	Documento L2 entregado; Product Backlog con al menos 12 historias de usuario, cada una con criterios de aceptación y prioridad.
H4	9	Diagrama de clases (5 a 8 clases o más) y al menos 3 diagramas de secuencia versionados en el repositorio y entregados en L3.
H5	12	Diagramas de arquitectura y despliegue entregados en L4; el script de la prueba técnica lee CPU, RAM, disco y uptime en una VM Windows.

Hito	Semana	Criterio de verificación
H6	13	Una VM con el agente instalado envía métricas a la API y aparecen en el dashboard.
H7	14	Desde el panel se ordena reiniciar una VM, la VM se reinicia y la acción queda en la auditoría; prototipo entregado en L5.
H8	15	Plan de pruebas con casos trazados a cada RF implementado, ejecutados y con resultado registrado; documento integrado entregado en L6.
H9	17	Presentación final realizada con demostración del prototipo en vivo.

Tabla 11. Hitos del proyecto.

5.4 Roles del equipo

Integrante	Rol principal	Responsabilidades
Felipe Cifuentes	Product Owner / Desarrollo – agente y API	Prioriza el backlog, valida alcance y requerimientos; lidera la prueba técnica del agente y el backend.
Paula Rivas	Scrum Master / Desarrollo – panel web y pruebas	Coordina reuniones, controla plazos y riesgos, gestiona entregas en UVirtual; lidera la interfaz y el plan de pruebas.

Tabla 12. Roles preliminares. Todos los integrantes participan en todas las fases y deben poder explicar cualquier parte del proyecto.

6. Identificación preliminar de riesgos

Se usan las categorías de riesgo de Sommerville (tecnología, personas, organizacionales, herramientas, requerimientos y estimación). La probabilidad se clasifica en baja, moderada o alta; el efecto en insignificante, tolerable, serio o catastrófico. La planificación de riesgos se refinará en la actividad de la semana 5.

ID	Riesgo	Categoría	Prob.	Efecto	Estrategia / mitigación
R1	La temperatura y otros sensores no son accesibles de forma uniforme en Windows (WMI no los expone en muchos equipos).	Tecnología	Alta	Tolerable	Validarlo en la prueba técnica (sem. 10-11); usar una librería alternativa (LibreHardwareMonitor) o marcar el dato como "no disponible".
R2	El alcance crece (formateo, escritorio remoto, más métricas) y no alcanza el tiempo para el prototipo.	Requerimientos	Alta	Serio	Backlog priorizado; acciones críticas definidas como simuladas; todo cambio de alcance se aprueba en la planificación del sprint.
R3	Se subestima el esfuerzo de instalar el agente como servicio protegido contra desinstalación.	Estimación	Moderada	Serio	Prueba técnica temprana; plan B: instalación con permisos de administrador y servicio estándar de Windows, documentando la limitación.
R4	Menor disponibilidad de integrantes por carga académica (PEP, otros ramos) o poca experiencia con React.	Personas	Moderada	Serio	Tareas distribuidas según fortalezas, trabajo en parejas en tareas nuevas y holgura antes de cada entrega.
R5	Cambios en el calendario académico (paros, feriados, suspensión de clases) reducen las semanas disponibles.	Organizacional	Moderada	Serio	Adelantar trabajo en los sprints iniciales y priorizar el backlog para recortar funciones de menor valor si es necesario.

Tabla 13. Riesgos preliminares del proyecto.